

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: df = pd.read_csv("Student_Performance.csv")
```

```
In [3]: df.head()
```

Out[3]:

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index
0	7	99	Yes	9	1	91.0
1	4	82	No	4	2	65.0
2	8	51	Yes	7	2	45.0
3	5	52	Yes	5	2	36.0
4	7	75	No	8	5	66.0

```
In [4]: df_backup = df.copy()
```

```
In [5]: df
```

Out[5]:

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced	Performance Index
0	7	99	Yes	9	1	91.0
1	4	82	No	4	2	65.0
2	8	51	Yes	7	2	45.0
3	5	52	Yes	5	2	36.0
4	7	75	No	8	5	66.0
...	...	...	...	...	...	...
9995	1	49	Yes	4	2	23.0
9996	7	64	Yes	8	5	58.0
9997	6	83	Yes	8	5	74.0
9998	9	97	Yes	7	0	95.0
9999	7	74	No	8	1	64.0

10000 rows × 6 columns

```
In [6]: df.isna().sum() # for missing values
```

```
Out[6]: Hours Studied      0
        Previous Scores    0
        Extracurricular Activities  0
        Sleep Hours        0
        Sample Question Papers Practiced  0
        Performance Index    0
        dtype: int64
```

```
In [7]: df.shape
```

Out[7]: (10000, 6)

```
In [8]: df.describe()
```

Out[8]:

	Hours Studied	Previous Scores	Sleep Hours	Sample Question Papers Practiced	Performance Index
count	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000
mean	4.992900	69.445700	6.530600	4.583300	55.224800
std	2.589309	17.343152	1.695863	2.867348	19.212558
min	1.000000	40.000000	4.000000	0.000000	10.000000
25%	3.000000	54.000000	5.000000	2.000000	40.000000
50%	5.000000	69.000000	7.000000	5.000000	55.000000
75%	7.000000	85.000000	8.000000	7.000000	71.000000
max	9.000000	99.000000	9.000000	9.000000	100.000000

```
In [9]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Hours Studied                        10000 non-null  int64
1   Previous Scores                      10000 non-null  int64
2   Extracurricular Activities          10000 non-null  str
3   Sleep Hours                         10000 non-null  int64
4   Sample Question Papers Practiced    10000 non-null  int64
5   Performance Index                   10000 non-null  float64
dtypes: float64(1), int64(4), str(1)
memory usage: 468.9 KB
```

```
In [10]: df.duplicated().sum()
```

Out[10]: np.int64(127)

```
In [11]: df.drop_duplicates(inplace=True)
```

```
In [12]: x = df.iloc[:, :-1]
x
# to select the features
```

Out[12]:

	Hours Studied	Previous Scores	Extracurricular Activities	Sleep Hours	Sample Question Papers Practiced
0	7	99	Yes	9	1
1	4	82	No	4	2
2	8	51	Yes	7	2
3	5	52	Yes	5	2
4	7	75	No	8	5
...	...	...	...	...	...
9995	1	49	Yes	4	2
9996	7	64	Yes	8	5
9997	6	83	Yes	8	5
9998	9	97	Yes	7	0
9999	7	74	No	8	1

9873 rows × 5 columns

```
In [13]: y = df.iloc[:, -1]
y
# to select the target
```

Out[13]:

0	91.0
1	65.0
2	45.0
3	36.0
4	66.0
...	
9995	23.0
9996	58.0
9997	74.0
9998	95.0
9999	64.0

Name: Performance Index, Length: 9873, dtype: float64

```
In [14]: pip install scikit-learn
```

Requirement already satisfied: scikit-learn in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (1.8.0)
 Requirement already satisfied: numpy>=1.24.1 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (2.4.5)
 Requirement already satisfied: scipy>=1.10.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (1.17.1)
 Requirement already satisfied: joblib>=1.3.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (1.5.3)
 Requirement already satisfied: threadpoolctl>=3.2.0 in .\AppData\Local\Python\pythoncore-3.14-64\Lib\site-packages (from scikit-learn) (3.6.0)
 Note: you may need to restart the kernel to use updated packages.

In [15]: `from sklearn.model_selection import train_test_split`

In [16]: `x_train , x_test , y_train , y_test = train_test_split(x , y , test_size=0.17 , sh`
to select the mounts of train and test

In [17]: `print(f"the shape of x is train {x_train.shape}")`
`print(f"the shape of x is test {x_test.shape}")`
`print(f"the shape of y is train {y_train.shape}")`
`print(f"the shape of y is test {y_test.shape}")`

the shape of x is train (8194, 5)
 the shape of x is test (1679, 5)
 the shape of y is train (8194,)
 the shape of y is test (1679,)

In [18]: `from sklearn.preprocessing import LabelEncoder` *# To convert into 0 , 1*

In [19]: `encoder = LabelEncoder()`

In [20]: `x_train["Extracurricular Activities"] = encoder.fit_transform(x_train["Extracurricu`
To convert into 0 , 1

In [21]: `x_train["Extracurricular Activities"]`

Out[21]: 4751 0
 5896 1
 5331 0
 1367 1
 1180 1
 ..
 9969 1
 4881 0
 1480 0
 4749 1
 1069 0
 Name: Extracurricular Activities, Length: 8194, dtype: int64

In [22]: `x_test["Extracurricular Activities"] = encoder.fit_transform(x_test["Extracurricula`

In [23]: `x_test["Extracurricular Activities"]`

```
Out[23]: 9187    1
          7471    0
          9193    1
          4361    0
          8180    0
          ..
          8744    1
          7386    1
          8561    1
          5439    1
          3052    0
          Name: Extracurricular Activities, Length: 1679, dtype: int64
```

```
In [24]: from sklearn.preprocessing import MinMaxScaler
```

```
In [25]: scaler = MinMaxScaler()
```

```
In [26]: x_train["Previous Scores"] = scaler.fit_transform(x_train[["Previous Scores"]])
          # To convert it
```

```
In [27]: x_train["Previous Scores"]
```

```
Out[27]: 4751    0.135593
          5896    0.067797
          5331    0.084746
          1367    0.389831
          1180    0.491525
          ...
          9969    0.135593
          4881    0.593220
          1480    0.627119
          4749    0.372881
          1069    0.627119
          Name: Previous Scores, Length: 8194, dtype: float64
```

```
In [28]: x_test["Previous Scores"] = scaler.transform(x_test[["Previous Scores"]])
          x_test["Previous Scores"]
```

```
Out[28]: 9187    0.152542
          7471    0.135593
          9193    0.491525
          4361    0.135593
          8180    0.220339
          ...
          8744    0.288136
          7386    0.915254
          8561    0.762712
          5439    0.203390
          3052    0.169492
          Name: Previous Scores, Length: 1679, dtype: float64
```

```
In [29]: from sklearn.linear_model import LinearRegression
```

```
In [30]: reg = LinearRegression()
```

```
In [31]: reg.fit(x_train , y_train)
```

```
Out[31]:
```

▼ LinearRegression ⓘ ?

► Parameters

```
In [32]: reg.coef_ # Weights "W"
```

```
Out[32]: array([ 2.85223268, 60.05071941,  0.64250256,  0.47614306,  0.19493972])
```

```
In [33]: reg.intercept_ # Bias "b"
```

```
Out[33]: np.float64(6.683451000244787)
```

```
In [34]: y_hat = reg.predict(x_test)
```

```
In [35]: y_hat
```

```
Out[35]: array([26.58466182, 41.92907042, 57.78736191, ..., 78.74284407,
                28.12339541, 40.37752141], shape=(1679,))
```

```
In [36]: print(f"The Score of the model is : {reg.score(x_test , y_test)*100}%")
# Accuracy
```

The Score of the model is : 98.9192358299518%

```
In [37]: from sklearn.metrics import mean_squared_error , mean_absolute_error
```

```
In [38]: mse = mean_squared_error(y_test , y_hat)
mse
```

```
Out[38]: 4.177833386221788
```

```
In [39]: mae = mean_absolute_error(y_test , y_hat)
mae
```

```
Out[39]: 1.6277728200004749
```

```
In [40]: summary = {
    "Target" : y_test ,
    "Prediction" : y_hat ,
    "Error": y_test - y_hat
}
summary = pd.DataFrame(summary)
```

```
In [41]: summary
```

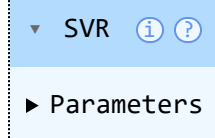
Out[41]:

	Target	Prediction	Error
9187	23.0	26.584662	-3.584662
7471	37.0	41.929070	-4.929070
9193	60.0	57.787362	2.212638
4361	41.0	43.266610	-2.266610
8180	26.0	26.902371	-0.902371
...	...	...	...
8744	38.0	37.774305	0.225695
7386	75.0	73.836900	1.163100
8561	78.0	78.742844	-0.742844
5439	28.0	28.123395	-0.123395
3052	42.0	40.377521	1.622479

1679 rows × 3 columns

In [42]: `from sklearn.svm import SVR # Support Vector Regressor`In [43]: `reg2 = SVR()`In [44]: `reg2.fit(x_train , y_train)`

Out[44]:

In [45]: `y_hat2 = reg2.predict(x_test)`In [46]: `print(f"the score of the model is : {reg2.score(x_train , y_train)*100}%")`

the score of the model is : 89.98051650104729%

In [47]: `print(f"The Score of the model is : {reg2.score(x_test , y_test)*100}%")`

The Score of the model is : 89.84904874262465%

In [48]: `score2 = reg2.score(x_test , y_test)*100`In [49]: `score1=reg.score(x_test , y_test)*100`In [50]: `scores = {
 "Models" : ["Linear Regression" , "Support Vector Regressor"] ,
 "Score" : [score1 , score2]
}`

```
scores = pd.DataFrame(scores)
scores
```

Out[50]:

	Models	Score
0	Linear Regression	98.919236
1	Support Vector Regressor	89.849049

```
In [51]: from sklearn.ensemble import RandomForestRegressor
```

```
In [52]: rf_reg = RandomForestRegressor(n_estimators=100 , random_state=42)
```

```
In [53]: rf_reg.fit(x_train , y_train)
```

Out[53]:

▼ RandomForestRegressor ⓘ ?

► Parameters

```
In [54]: score3 =rf_reg.score(x_test , y_test)*100
```

```
In [55]: print(f"Random Forest Score : {score3}")
```

Random Forest Score : 98.55311953843567

```
In [56]: from sklearn.ensemble import GradientBoostingRegressor
```

```
In [57]: gb_reg = GradientBoostingRegressor()
```

```
In [58]: gb_reg.fit(x_train, y_train)
```

Out[58]:

▼ GradientBoostingRegressor ⓘ ?

► Parameters

```
In [59]: score4 = gb_reg.score(x_test, y_test) * 100
```

```
In [60]: print(f"Gradient Boosting Score: {score4}")
```

Gradient Boosting Score: 98.8481466855836

```
In [61]: scores = {
    "Models" : ["Linear Regression", "Support Vector Regressor", "Random Forest", "
    "Score" : [score1, score2, score3, score4]
}

scores_df = pd.DataFrame(scores)
scores_df
```

Out[61]:

	Models	Score
0	Linear Regression	98.919236
1	Support Vector Regressor	89.849049
2	Random Forest	98.553120
3	Gradient Boosting	98.848147

In []: